

Project information

Created on
February 24, 2026

Update file README.md

n36 authored 1 minute ago

Name	Last commit	Last update
README.md	Update file README.md	1 minute ago

README.md

Как устроена система бэкапов (для нетехнаря)

Обзор для нетехнаря: как всё устроено

Зачем это нужно. У вас есть много «стендов» (окружений) с базами данных. Чтобы не потерять данные, сервер **регулярно** делает их копии (бэкапы) и складывает на диск. Одновременно он **сверяет** список стендов с биллингом и с тем, что реально есть в кластере (Kubernetes), и шлёт отчёты в Telegram.

Как это запускается. На сервере настроен **планировщик по времени** (cron): одни задачи — каждые 10 минут, другие — раз в день в 03:15, третьи — каждые 6 часов. Никто вручную ничего не нажимает: всё срабатывает само.

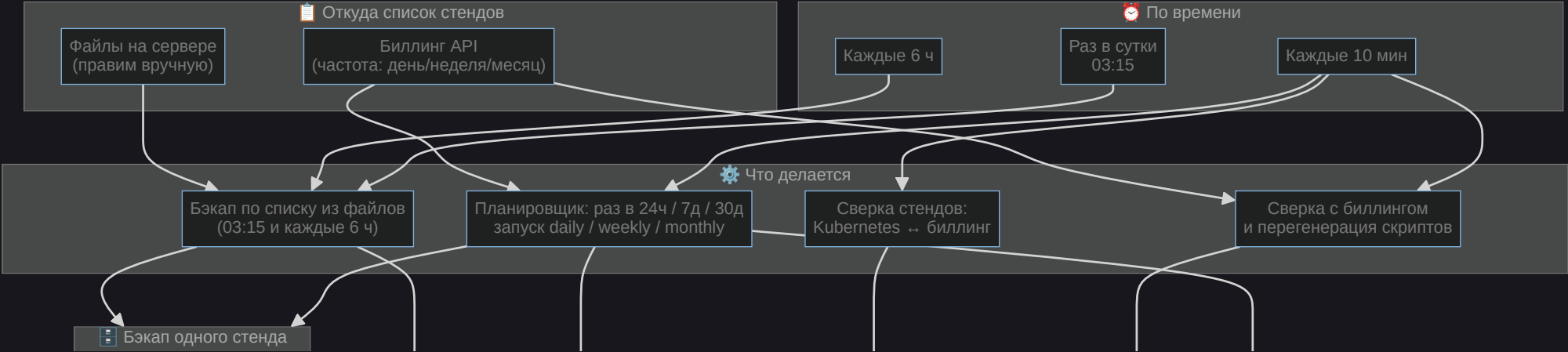
Два способа решить, кого бэкапить.

- Список в файлах на сервере** — туда вручную прописаны имена стендов. По этому списку идут «ежедневный» бэкап (раз в сутки в 03:15) и бэкап «каждые 6 часов». Чтобы добавить или убрать стенд — нужно править эти файлы.
- Список из биллинга** — сервер каждые 10 минут спрашивает биллинг: какие стенды есть и с какой частотой их бэкапить (раз в день / раз в неделю / раз в месяц). По этому ответу он **сам формирует** скрипты и **сам же** запускает их по расписанию: раз в 24 часа — ежедневный, раз в 7 дней — еженедельный, раз в 30 дней — ежемесячный. Расписание хранится в файле состояния и в CSV-журнале на сервере.

Синхронизация списка стендов. Отдельный процесс каждые 10 минут смотрит неймспейсы в Kubernetes и биллинг: для новых неймспейсов создаёт стенды в биллинге, для исчезнувших — помечает на удаление. Так список в биллинге не расходится с реальностью.

Что в итоге происходит с данными. Для каждого стенда из списка сервер подключается к его базе (PostgreSQL в Yandex Cloud), делает дамп, сжимает и сохраняет файл на диск. Параллельно пишет события в ELK и шлёт в Telegram: «бэкап запущен», «успешно столько-то стендов», «упали такие-то» — с деталями.

Одна картинка всего процесса:



Кратко. Планировщик по времени запускает процессы. Список стендов берётся либо из файлов на сервере, либо из биллинга. Сам бэкап — всегда один и тот же: дамп БД → сжатый файл на диск. Результаты: диск, ELK, Telegram.

Два независимых контура

Есть два способа решить, «кого бэкапить»:

1. **Список зашит в скриптах на сервере** — его меняют вручную (редактирование файлов). По этому списку крутятся **ежедневный** и **каждые 6 часов** бэкапы.
2. **Список берётся из API биллинга** — в биллинге у каждого стенда есть настройка «как часто бэкапить» (daily/weekly/monthly). Сервер раз в 10 минут спрашивает биллинг, получает актуальный список и **перегенерирует скрипты** бэкапов. Кто в биллинге — того и бэкапят по сгенерированным скриптам.

Плюс отдельно: **синхронизация списка стендов** — сервер смотрит неймспейсы в Kubernetes и создаёт/помечает на удаление стенды в биллинге, чтобы список в биллинге совпадал с тем, что реально есть в кластере.

Кто что запускает (по времени)

Когда	Что запускается	Откуда список стендов	Результат
Каждые 10 мин	Синхронизация с биллингом	API биллинга	Обновляются скрипты бэкапов (daily/weekly/monthly), список стендов в биллинге сверяется с k8s
Каждые 10 мин	Тот же процесс	—	Если прошло ≥6 ч с прошлого запуска — выполняется периодический бэкап по сгенерированному daily-скрипту
Раз в сутки 03:15	Ежедневный бэкап	Список в файле на сервере	Бэкапы только тех стендов, что прописаны в <code>daily_backups.sh</code>
Каждые 6 ч	Бэкап «каждые 6 часов»	Список в файле на сервере	Бэкапы только тех стендов, что прописаны в <code>6_h_backups.sh</code>

Итого: **один процесс по крону каждые 10 минут** делает и синхронизацию с биллингом, и (при необходимости) периодический 6h-бэкап по списку из биллинга. Отдельно по крону в 03:15 и каждые 6 ч запускаются **другие** скрипты с **жёстко прописанным** списком стендов.

Откуда что берётся

- Список для «ручных» бэкапов (раз в день и каждые 6 ч)**
Прописан в двух файлах на сервере. Чтобы добавить/убрать стенд — нужно отредактировать эти файлы.
- Список для бэкапов по биллингу (daily/weekly/monthly и периодический 6h)**
Приходит из **биллинга** (API). Сервер запрашивает список стендов и для каждого — частоту бэкапа. По этим данным он **генерирует** три скрипта (daily, weekly, monthly) и при необходимости запускает один из них (периодический 6h = тот же список, что и daily).
- Актуальность «кто есть стенд» в биллинге**
Сервер смотрит **неймспейсы в Kubernetes** и синхронизирует с биллингом: для новых неймспейсов создаёт стенды, для исчезнувших — помечает на удаление (если не защищены).

Куда уходят результаты

- Файлы бэкапов** — на диск сервера (каталог бэкапов, сжатые дампы).
- События и логи** — в ELK (для аналитики и дашбордов).
- Уведомления** — в Telegram (запуск бэкапа, успех/ошибки по стендам, синхронизация).

Кто запускает скрипты, которые генерирует talk_to_elk?

talk_to_elk только **создаёт** три файла: `billing_backups_daily.sh`, `billing_backups_weekly.sh`, `billing_backups_monthly.sh`. Их кто-то должен **запустить**.

- Кто запускает:** сам **talk_to_elk.py**. Он вызывается по крону **каждые 10 минут** без аргументов (`run_billing_sync.sh` → `talk_to_elk.py`). При каждом таком запуске он:
 - Синхронизируется с биллингом и перегенерирует скрипты.
 - Смотрит: прошло ли **≥ 6 часов** с последнего запуска бэкапа.
 - Если да — **сам же** запускает сгенерированный **billing_backups_daily.sh** (это и есть «периодический бэкап 6h»).
- Что с daily/weekly/monthly по отдельности:** в коде есть режимы `--execute-daily`, `--execute-weekly`, `--execute-monthly` (запуск соответствующего скрипта), но в **кроне** и **systemd** на сервере **никто их не вызывает**. То есть:
 - billing_backups_daily.sh** — реально выполняется только изнутри talk_to_elk (раз в 6 ч).
 - billing_backups_weekly.sh** и **billing_backups_monthly.sh** — по текущей настройке **ничем по расписанию не запускаются** (их можно запустить вручную или добавить в cron вызовы типа `talk_to_elk.py --execute-weekly` и т.п.).

Итого: **заставляет выполняться** сгенерированные скрипты **логика внутри talk_to_elk** при его обычном запуске каждые 10 минут (планировщик по CSV/state).

Планировщик и CSV (обновлено):

- В каталоге состояния (/var/lib/billing_backup_sync/) лежат:
 - backup_schedule_state.json** — время последнего запуска по типам: **daily** , **weekly** , **monthly** .
 - backup_schedule.csv** — журнал запусков: каждая строка = один запуск (script, ran_at_iso, exit_code, stands_processed, total_stands).
- При каждом запуске без флага talk_to_elk проверяет: прошло ли **24 ч** с последнего daily, **7 дней** с weekly, **30 дней** с monthly. Если да — запускает соответствующий скрипт, пишет в state и в CSV, шлёт в Telegram: «📅 Запущен ежедневный бэкап», «📅 Запущен еженедельный бэкап», «📅 Запущен ежемесячный бэкап» и затем детальный отчёт (успех/ошибки по стендам), как и раньше.

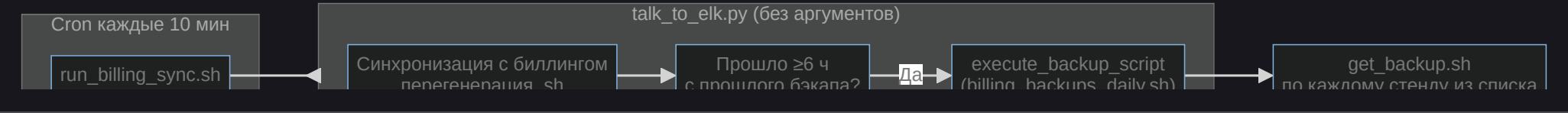


Схема (Mermaid): общая парадигма

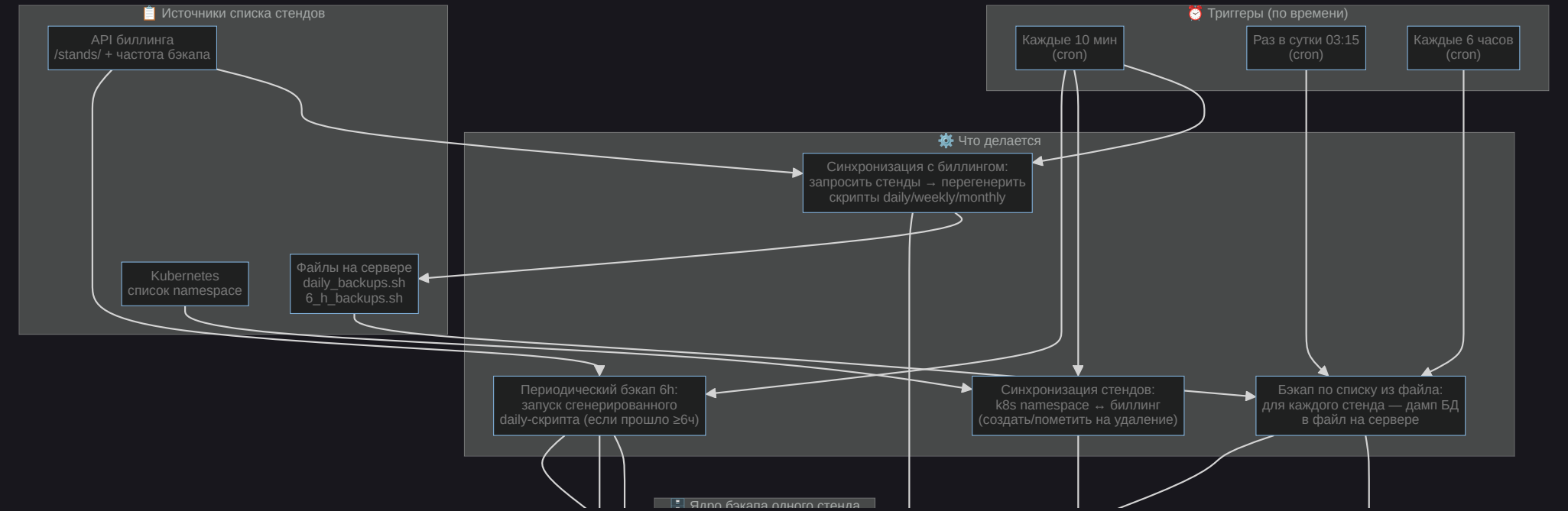
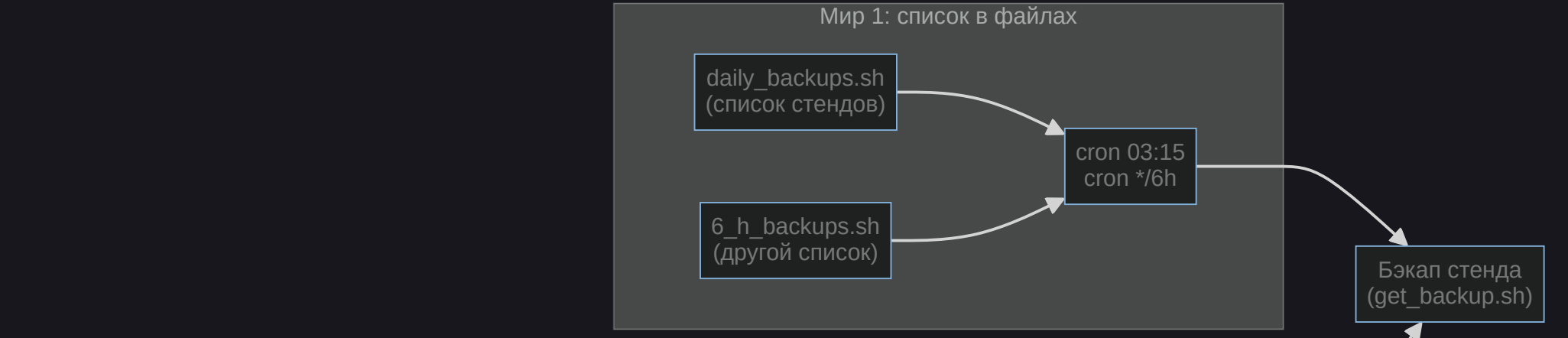


Схема: два «мира» списков



Кратко для технаря

- Парадигма:** по расписанию (cron) запускаются процессы; они либо **берут список стендов из файлов**, либо **из API биллинга**, либо **синхронизируют список в биллинге с k8s**.
- Бэкап одного стенда** всегда один и тот же: найти namespace и пароль в k8s → дамп БД в облако → сохранить сжатый файл на диск.
- Два независимых списка:** (1) зашит в двух скриптах — для бэкапов в 03:15 и каждые 6 ч; (2) из биллинга — для сгенерированных скриптов; планировщик (state + CSV) запускает daily раз в 24 ч, weekly раз в 7 д, monthly раз в 30 д.
- Результаты:** файлы на диске, события в ELK, уведомления в Telegram.